

Assignment -05_HPC_LAB_U23AI060

Program_01:

```
C 01_p.c
1 #include <stdio.h>
2 #include <math.h>
3 #include <omp.h>
4
5 int main() {
6     int n = 100000;
7     int arr[n];
8
9     for (int i = 0; i < n; i++) {
10         arr[i] = i * (i % 5);
11     }
12
13     double log_product_atomic = 0.0;
14     double log_product_critical = 0.0;
15
16     #pragma omp start, end;
17
18     start = omp_get_wtime();
19     // Atomic Approach
20
21     #pragma omp parallel for
22     for (int i = 0; i < n; i++) {
23         double val = log(arr[i]);
24
25         #pragma omp atomic
26         log_product_atomic += val;
27     }
28
29     end = omp_get_wtime();
30     double time_atomic = end - start;
31
32     // Critical Section Approach
33     start = omp_get_wtime();
34
35     #pragma omp parallel for
36     for (int i = 0; i < n; i++) {
37         double val = log(arr[i]);
38
39         #pragma omp critical
40         {
41             log_product_critical += val;
42         }
43     }
44
45     end = omp_get_wtime();
46     double time_critical = end - start;
47
48     printf("Log Product (Atomic) : %f\n", log_product_atomic);
49     printf("Time (Atomic) : %f seconds\n", time_atomic);
50
51     printf("Log Product (Critical) : %f\n", log_product_critical);
52     printf("Time (Critical) : %f seconds\n", time_critical);
53
54     return 0;
55 }
```

output:

```
(base) gagan747@gaganLaptop:/mnt/d/Semester_06/HPC/LAB/06_lab_$ ./q1
Log Product (Atomic) : 957498.348564
Time (Atomic) : 0.094944 seconds
Log Product (Critical) : 957498.348564
Time (Critical) : 0.283989 seconds
```

Analysis:

Analysis:

In this program, the product of array elements is computed using two synchronization methods: atomic and critical. The atomic directive provides fine-grained synchronization with lower overhead, while the critical directive enforces mutual exclusion on an entire block, increasing waiting time.

For small workloads, critical may appear faster due to reduced contention, but for larger workloads, atomic is generally more efficient due to better parallelism.

Program_02:

```
C 02_p.c
1 #include <stdio.h>
2 #include <omp.h>
3
4 int main() {
5     int n = 100;
6     int arr[n];
7
8     // Initialization
9     for (int i = 0; i < n; i++) {
10         arr[i] = i + 1;
11     }
12
13     long long global_sum = 0;
14     double global_product = 1.0;
15
16     // Parallel Region
17     #pragma omp parallel
18     {
19         long long local_sum = 0;
20         double local_product = 1.0;
21
22         // Divide work
23         #pragma omp for
24         for (int i = 0; i < n; i++) {
25             local_sum += arr[i];
26             local_product *= arr[i];
27         }
28
29         // Named critical for sum
30         #pragma omp critical(global_sum_section)
31         {
32             global_sum += local_sum;
33         }
34
35         // Named critical for product
36         #pragma omp critical(global_product_section)
37         {
38             global_product *= local_product;
39         }
40     }
41
42     // Output
43     printf("Global Sum : %lld\n", global_sum);
44     printf("Global Product : %e\n", global_product);
45
46     return 0;
47 }
```

Output:

```
(base) gagan747@gaganLaptop:/mnt/d/Semester_06/HPC/LAB/06_lab_$ ./p2
Global Sum : 5050
Global Product : 9.332622e+157
```

Analysis:

Analysis:

This program demonstrates the use of multiple named critical sections to update independent shared variables (sum and product). By assigning different names to critical sections, separate locks are created, allowing threads to update sum and product concurrently without unnecessary blocking. This improves parallel efficiency compared to using a single unnamed critical section.

Program_03:

```
C 03_p.c
1  #include <stdio.h>
2  #include <omp.h>
3
4  #define N 100
5  #define BUCKETS 4
6
7  int main() {
8      int arr[N];
9      for (int i = 0; i < N; i++) arr[i] = i + 1;
10
11      // ----- Global Lock -----
12      omp_lock_t lock;
13      omp_init_lock(&lock);
14
15      long long sum_global = 0;
16      double start = omp_get_wtime();
17
18      #pragma omp parallel for
19      for (int i = 0; i < N; i++) {
20          omp_set_lock(&lock);
21          sum_global += arr[i];
22          omp_unset_lock(&lock);
23      }
24
25      double time_global = omp_get_wtime() - start;
26      omp_destroy_lock(&lock);
27
28      // ----- Bucket Lock -----
29      omp_lock_t bucket_locks[BUCKETS];
30      long long bucket_sum[BUCKETS] = {0};
31
32      for (int i = 0; i < BUCKETS; i++)
33          omp_init_lock(&bucket_locks[i]);
34
35      start = omp_get_wtime();
36
37      #pragma omp parallel for
38      for (int i = 0; i < N; i++) {
39          int b = i % BUCKETS;
40
41          omp_set_lock(&bucket_locks[b]);
42          bucket_sum[b] += arr[i];
43          omp_unset_lock(&bucket_locks[b]);
44      }
45
46      long long sum_bucket = 0;
47      for (int i = 0; i < BUCKETS; i++) {
48          sum_bucket += bucket_sum[i];
49          omp_destroy_lock(&bucket_locks[i]);
50      }
51
52      double time_bucket = omp_get_wtime() - start;
53
54      printf("Global Lock Sum   : %lld, Time: %f\n", sum_global, time_global);
55      printf("Bucket Lock Sum    : %lld, Time: %f\n", sum_bucket, time_bucket);
56
57      return 0;
58 }
```

Output :

```
(base) gagan747@gaganLaptop: /mnt/d/Semester_06/HPC/LAB/06_lab_$ ./p3
Global Lock Sum   : 5050, Time: 0.012516
Bucket Lock Sum   : 5050, Time: 0.006561
```

Analysis:

Analysis:

Global lock causes high contention as all threads compete for a single lock, reducing parallelism. Bucket locking reduces contention by allowing multiple threads to work on different partitions simultaneously. However, managing multiple locks introduces slight overhead. Overall, bucket locking improves performance for larger workloads.

Program_04:

```
C 04_p.c
1  #include <stdio.h>
2  #include <omp.h>
3
4  #define N 4
5
6  int main() {
7      int A[N][N], x[N], y[N] = {0};
8
9      // Initialization (small values)
10     for (int i = 0; i < N; i++) {
11         x[i] = i + 1;
12         for (int j = 0; j < N; j++) {
13             A[i][j] = j + 1;
14         }
15     }
16
17     #pragma omp parallel
18     {
19         #pragma omp single
20         {
21             for (int i = 0; i < N; i++) {
22                 #pragma omp task depend(out: y[i])
23                 {
24                     for (int j = 0; j < N; j++) {
25                         y[i] += A[i][j] * x[j];
26                     }
27                 }
28             }
29         }
30     }
31
32     printf("Result vector y:\n");
33     for (int i = 0; i < N; i++) {
34         printf("%d ", y[i]);
35     }
36     printf("\n");
37
38     return 0;
39 }
40
```

Output

```
(base) gagan747@gaganLaptop: /mnt/d/Semester_06/HPC/LAB/06_lab_$ ./p4
Result vector y:
30 30 30 30
```

Analysis:

Analysis:

This program performs a matrix-vector multiplication using OpenMP tasks with dependencies. Each task computes one element of the result vector y , and the `depend` clause ensures that tasks are executed in the correct order. The `single` directive ensures that only one thread creates the tasks, while the parallel region allows multiple threads to execute them concurrently.

Analysis:

The matrix-vector multiplication is decomposed into tasks, where each task computes one row of the result. The `depend` clause ensures proper data flow and avoids race conditions. This allows efficient parallel execution while maintaining correctness.

Program_05:

C 05_p.c

```
1  #include <stdio.h>
2  #include <omp.h>
3
4  #define N 400
5
6  int main() {
7      int A[N], B[N];
8
9
10     for (int i = 0; i < N; i++) {
11         A[i] = i + 1;
12         B[i] = (i * 10) + 1;
13     }
14
15     long long sum = 0, diff = 0;
16     double product = 1.0, division = 0.0;
17
18     #pragma omp parallel
19     {
20         #pragma omp single
21         {
22
23             #pragma omp task
24             {
25                 long long local_sum = 0;
26                 for (int i = 0; i < N; i++)
27                     local_sum += A[i] + B[i];
28
29                 #pragma omp critical
30                 sum += local_sum;
31             }
32
33             #pragma omp task
34             {
35                 long long local_diff = 0;
36                 for (int i = 0; i < N; i++)
37                     local_diff += (A[i] - B[i]);
38
39                 #pragma omp critical
40                 diff += local_diff;
41             }
42
43             /
44             #pragma omp task
45             {
46                 double local_product = 1.0;
47                 for (int i = 0; i < 20; i++)
48                     local_product *= (A[i] * B[i]);
49
50                 #pragma omp critical
51                 product *= local_product;
52             }
53
54
55             #pragma omp task
56             {
57                 double local_div = 0.0;
58                 for (int i = 0; i < N; i++)
59                     local_div += (double)A[i] / B[i];
60
61                 #pragma omp critical
62                 division += local_div;
63             }
64
65
66             #pragma omp taskwait
67
68
69             printf("Sum      : %lld\n", sum);
70             printf("Difference : %lld\n", diff);
71             printf("Product  : %e\n", product);
72             printf("Division : %f\n", division);
73         }
74     }
75
76     return 0;
77 }
78
```

Output:

```
(base) gagan747@gaganLaptop: /mnt/d/Semester_06_/HPC/LAB/06_lab_$ ./p5
Sum      : 82400
Difference : 78000
Product  : 3.283591e+31
Division  : 23245.952381
(base) gagan747@gaganLaptop: /mnt/d/Semester_06_/HPC/LAB/06_lab_$
```

Analysis:

Analysis:

The program uses multiple tasks to perform independent computations in parallel. Race conditions are avoided using local variables and critical sections. Task synchronization is ensured using taskwait, guaranteeing that results are printed only after all tasks complete.